

COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>SQE</i>
<i>Instructor</i>	<i>Mam Ambreen</i>
<i>Lab</i>	<i>Project</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-037</i> <i>FA24-BSE-042</i>	<i>M Hammad Khalid</i> <i>Sami Ullah</i>

Project title : Secure Exam System

1. Purpose:

The purpose of this project is to develop a Secure Exam System that protects exam papers from unauthorized modification or tampering. The project uses the SHA-256 hashing algorithm to verify the integrity of exam papers.

Hashing generates a unique fixed-length value for data. If even one character changes in the exam paper, the generated hash value becomes completely different. This helps detect unauthorized changes.

2. Objectives

The main objectives of this project are:

- To secure exam papers from unauthorized modifications.
- To implement SHA-256 hashing in Python.
- To verify the integrity of exam papers.
- To understand the practical implementation of information security concepts.
- To prevent exam paper leakage and tampering.

3. Selected Encryption Technique

SHA-256 Hashing Algorithm

SHA-256 stands for Secure Hash Algorithm 256-bit. It is a cryptographic hashing algorithm widely used in cybersecurity and data protection systems.

The algorithm converts input data into a fixed 256-bit hash value. The generated hash is unique for every input.

Example:

Input:
Exam starts at 9 AM

SHA-256 Hash:
a6c1e0f91d8f0dce3a...

Even a small change in the text creates a completely different hash value.

4. Real-World Application

SHA-256 is used in many real-world systems such as:

- Password protection systems
- Digital signatures
- Blockchain technology
- File integrity verification
- Banking systems
- Secure communication systems

In this project, SHA-256 is used to protect exam papers from unauthorized changes.

5. Advantages and Disadvantages

Advantages

- Fast and secure
- Detects file modifications easily
- Widely accepted security standard
- Simple implementation in Python
- Produces unique hash values

Disadvantages

- Hashing alone does not encrypt data
- If both file and hash are changed, tampering may not be detected
- Cannot recover original data from hash

6. System Working

➤ Hash Key Generation

In this project, SHA-256 hashing is used to generate a unique hash value for the exam paper. The system converts the exam content into a fixed-length hash using the `hashlib` library in Python. This hash acts like a digital fingerprint of the paper. Even a small change in the paper creates a completely different hash value.

➤ **Exam Paper Security Using Hashing**

The exam paper is secured by generating its SHA-256 hash before storing or sharing it. The hash value helps protect the integrity of the paper and ensures that no unauthorized modifications are made. This method is commonly used in real-world security systems for data verification.

➤ **Verification of Exam Paper**

During verification, the system again generates a hash for the received exam paper. The newly generated hash is compared with the original stored hash. If both hashes are the same, the paper is original and unchanged. If the hashes are different, it means the paper has been modified or tampered with.

➤ **Detection of Tampering**

The system can easily detect unauthorized changes in the exam paper because SHA-256 produces different hash values for different inputs. Even changing one letter or symbol in the paper creates a completely new hash. This helps maintain the confidentiality and integrity of examination documents.

➤ **Overall Security of the System**

The Secure Exam System improves the security of exam papers by preventing hidden modifications and paper leakage. Hashing provides integrity checking, which is important in educational systems. The project demonstrates the practical implementation of information security concepts using Python programming.

Code:

```
import hashlib
import os
import json

students_db = {}
exams_db = {}
submissions_db = {}

def register_student(student_id, name, password):
    salt = os.urandom(16)
    hashed = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
    students_db[student_id] = {
        "name": name,
        "salt": salt.hex(),
        "password_hash": hashed.hex()
    }
    print(f"Student '{name}' registered successfully.")

def login_student(student_id, password):
    if student_id not in students_db:
        print("Student not found.")
        return False
    student = students_db[student_id]
    salt = bytes.fromhex(student["salt"])
    new_hash = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
    if new_hash.hex() == student["password_hash"]:
        print(f"Login successful. Welcome, {student['name']}!")
        return True
    else:
        print("Incorrect password. Access denied.")
        return False

def upload_exam(exam_id, exam_content):
    exam_hash = hashlib.sha256(exam_content.encode()).hexdigest()
    exams_db[exam_id] = {
        "content": exam_content,
        "original_hash": exam_hash
    }
    print(f"Exam '{exam_id}' uploaded.")
    print(f"Original Hash: {exam_hash}")

def verify_exam(exam_id):
    if exam_id not in exams_db:
        print("Exam not found.")
        return False
    exam = exams_db[exam_id]
    current_hash = hashlib.sha256(exam["content"].encode()).hexdigest()
    if current_hash == exam["original_hash"]:
        print("Exam Integrity CHECK PASSED - Exam is original and untampered.")
        return True
    else:
        print("ALERT: Exam has been TAMPERED! Hash mismatch detected.")
        return False

def submit_answers(student_id, exam_id, answers):
    combined = student_id + exam_id + json.dumps(answers, sort_keys=True)
    fingerprint = hashlib.sha256(combined.encode()).hexdigest()
    submissions_db[student_id + "_" + exam_id] = {
        "student_id": student_id,
        "exam_id": exam_id,
        "answers": answers,
        "fingerprint": fingerprint
    }
    print(f"Answers submitted by {student_id}.")
    print(f"Answer Sheet Fingerprint: {fingerprint}")

def verify_submission(student_id, exam_id):
    key = student_id + "_" + exam_id
    if key not in submissions_db:
        print("Submission not found.")
        return False
```

```

def submit_answers(student_id, exam_id, answers):
    combined = student_id + exam_id + json.dumps(answers, sort_keys=True)
    fingerprint = hashlib.sha256(combined.encode()).hexdigest()
    submissions_db[student_id + "_" + exam_id] = {
        "student_id": student_id,
        "exam_id": exam_id,
        "answers": answers,
        "fingerprint": fingerprint
    }
    print(f"Answers submitted by {student_id}.")
    print(f"Answer Sheet Fingerprint: {fingerprint}")

def verify_submission(student_id, exam_id):
    key = student_id + "_" + exam_id
    if key not in submissions_db:
        print("Submission not found.")
        return False
    sub = submissions_db[key]
    combined = student_id + exam_id + json.dumps(sub["answers"], sort_keys=True)
    current_fp = hashlib.sha256(combined.encode()).hexdigest()
    if current_fp == sub["fingerprint"]:
        print("Submission Integrity CHECK PASSED - Answers are original.")
        return True
    else:
        print("ALERT: Answer sheet has been MODIFIED after submission!")
        return False

```

Output:

```

PS C:\Users\HAMMAD\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HAMMAD\AppData\Local\Programs\Python\Python38\python.exe exam.py
--- Student Registration ---
Student 'Ali Hassan' registered successfully.
Student 'Sara Khan' registered successfully.

--- Student Login ---
Login successful. Welcome, Ali Hassan!
Incorrect password. Access denied.

--- Exam Upload ---
Exam 'EXAM_CSC232' uploaded.
Original Hash: 8381afb9db3b20076089d7c1bb333565d332c312899ccce3aa9d94b15204cd2c

--- Exam Integrity Verification ---
Exam Integrity CHECK PASSED - Exam is original and untampered.

--- Answer Submission ---
Answers submitted by S001.
Answer Sheet Fingerprint: 51c827f6c2b84ede2ff38e4f87390a9f3dbeab10ab2a010c7de30d264d4d219a

--- Submission Integrity Verification ---
Submission Integrity CHECK PASSED - Answers are original.

--- SHA Algorithm Comparison ---
MD5      (128 bits): 8b12689387dd028117dfe3e3b65e1794...
SHA1     (160 bits): 33434210d5eee94649802d9a348093b0c521be07...
SHA256   (256 bits): c6ebecf6d5c1a9e1dc1a8b35fecdd1c8c8a6654ce8f6d4...
SHA512   (512 bits): 155b2a1cf8e07382da73cd8a6fbdf4d31aa69e091590d...

=====
System secured with SHA-256 + PBKDF2 + Salt
=====
PS C:\Users\HAMMAD\AppData\Local\Programs\Microsoft VS Code>

```